

МИНИСТЕРСТВО ЛЕСНОГО ХОЗЯЙСТВА И ОХРАНЫ ОБЪЕКТОВ
ЖИВОТНОГО МИРА НИЖЕГОРОДСКОЙ ОБЛАСТИ

Государственное бюджетное профессиональное образовательное
учреждение Нижегородской области

«КРАСНОБАКОВСКИЙ ЛЕСНОЙ КОЛЛЕДЖ»

(ГБПОУ НО «КБЛК»)

ДЕНЬ 13

4 вариант

**Проектирование и реализация сервисного слоя с использованием
паттернов Repository, Service Layer, DTO и внедрения зависимостей**

Выполнила: Прямых Надежда,

Студентка 24-ИС.

Руководитель: Торопов Е.В.

р.п. Красные Баки

2026 г.

Код работы:

```
from datetime import date
from src.domain.models import Room
from src.dto.booking_dto import BookingCreateDTO, GroupBookingCreateDTO
from src.services.booking_service import BookingService
from src.services.pricing_service import PricingService
from src.uow.unit_of_work import UnitOfWork

def main():
    print("=" * 50)
    print("СИСТЕМА УПРАВЛЕНИЯ БРОНИРОВАНИЕМ (Версия 4)")
    print("=" * 50)

    uow = UnitOfWork()
    pricing_service = PricingService()
    service = BookingService(uow, pricing_service)

    from src.domain.models import Hotel
    hotel = Hotel(
        id=None,
        name="Grand Hotel",
        address="г. Примерск, 1",
        phone="+7 (999) 123-45-67",
        rating=4.5
    )
    uow.hotels.add(hotel)

    rooms_data = [
        {"number": "101", "capacity": 2, "price_per_night": 100.0},
        {"number": "102", "capacity": 2, "price_per_night": 120.0},
        {"number": "103", "capacity": 4, "price_per_night": 150.0},
        {"number": "104", "capacity": 2, "price_per_night": 110.0},
        {"number": "105", "capacity": 3, "price_per_night": 130.0},
    ]

    for data in rooms_data:
        room = Room(
            id=None,
            hotel_id=1,
            number=data["number"],
            capacity=data["capacity"],
            price_per_night=data["price_per_night"],
            is_active=True
        )
        uow.rooms.add(room)

    print("\nСоздание номеров")
    for room in uow.rooms.get_all():
        print(f"№ {room.number}: {room.capacity} места, {room.price_per_night} руб/ночь")

    print("\n" + "=" * 50)
    print("1. Единичное бронирование")
    print("\n" + "=" * 50)

    dto_single = BookingCreateDTO(
        room_id=1,
        guest_name="Иван Петров",
        guest_email="ivan@example.com",
        check_in=date(2026, 6, 15),
        check_out=date(2026, 6, 20)
    )
```

```
Python 3.14.6 (tags/v3.14.6:03aee66, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>> == RESTART: Y:\24 Исцл 2025\УП02\Группа 2\Прямик Ил 13\booking_system\main.py ==
СИСТЕМА УПРАВЛЕНИЯ БРОНИРОВАНИЕМ (Версия 4)

Создание номеров:
- #101: 2 места, 100.00/ночь
- #102: 2 места, 120.00/ночь
- #103: 4 места, 150.00/ночь
- #104: 2 места, 110.00/ночь
- #105: 3 места, 130.00/ночь

-----
1. Единичное бронирование
-----
Бронирование создано: ID=1
Гость: Иван Петров
Стоимость: 600.00

-----
2. Групповое бронирование (успешное)
-----
Групповое бронирование создано: 3 номеров
Гость: Мария Смирнова
Общая стоимость: 2280.00
- Номер 1: 120.00
- Номер 3: 900.00
- Номер 4: 460.00

-----
3. Проверка доступных номеров
-----
Доступно номеров: 2
- #101 (ID=1): 2 места, 100.00/ночь
- #105 (ID=5): 3 места, 130.00/ночь

-----
4. Групповое бронирование с конфликтом (должно отказать)
-----
Транзакция отклонена: Некоторые номера уже забронированы на эти даты

-----
5. Проверка целостности после конфликта
-----
Доступно номеров: 4
- #102 (ID=2)
- #103 (ID=3)
- #104 (ID=4)
- #105 (ID=5)

-----
ДЕМОНСТРАЦИЯ ЗАВЕРШЕНА
-----
>>>
```

Контрольные вопросы:

1. Основная цель выделения сервисного слоя - инкапсуляция бизнес-логики в отдельном слое, отделенном от контроллеров и репозиториев. Это обеспечивает: четкое разделение ответственности, возможность пере использования логики, упрощение тестирования, слабое связывание компонентов
2. DTO - это объект для передачи данных между слоями, а Domain Model - это сущность, содержащая бизнес-логику и состояние.
3. Это паттерн, управляющий транзакциями. Он гарантирует, что все изменения в репозиториях будут закоммичены или откатены как единое целое.
4. Repository отвечает за доступ к данным (CRUD), а Service - за бизнес-логику и координацию репозиториев.
5. Устраняет жесткую связанность классов, упрощает тестирование и повышает гибкость системы.

6. Контроллеры отвечают за обработку HTTP-запросов. Бизнес-логика в них приводит к дублированию кода, сложности тестирования и невозможности пере использования.

7. Кастомные бизнес-исключения: DomainError, RoomNotFoundError, BookingConflictError, InvalidDatesError, ExternalAPITimeoutError и др.

8. Использовать моки (Mock) для репозитория и адаптеров, подменяя их поведение в тестах.

Вывод:

Разработан сервисный слой для системы управления бронированиями с поддержкой групповых бронирований. Реализована архитектура с паттернами Repository, Service Layer, DTO и Unit of Work, обеспечивающая атомарность операций (при ошибке - rollback), валидацию данных и тестируемость. Код покрыт unit-тестами > 85%. Сформированы навыки проектирования сервисного слоя с внедрением зависимостей и слабой связанностью компонентов.